Universidad Católica de Oriente · Facultad de Ingeniería
Compiladores — Entrega Final · Semestre 2-2026

# Paisascript

Gramática ampliada final
Análisis léxico, sintáctico, semántico y traducción orientada por la sintaxis

**Lenguaje fuente:** Paisascript
**Lenguaje destino:** Gleam
**Integrantes:** Andres Felipe Velez · Angie Amaya Fernandez · Julian David Guarnizo Castrillon
**Repositorio:** github.com/JulianG08/Analizador-lexico
**App en vivo:** compiladorlex.streamlit.app
**Bonus:** Modalidad C — validación del código generado con IA y sugerencias de optimización

# 1. Gramática ampliada final

Este documento presenta la gramática completa de Paisascript con las tres capas de anotación que el proyecto fue acumulando entrega tras entrega. Las **producciones sintácticas originales** van en rojo y son idénticas a las de las entregas anteriores. Las **reglas semánticas** van sobre fondo ámbar. Las **reglas de traducción orientada por la sintaxis** van en azul.

`<produccion> ::= ...` — regla original de la gramática (producción sintáctica sin ampliar).

REGLA SEM-x — regla semántica: atributos, dominio de valores y acciones de validación.

**REGLA SDT-x** — regla de traducción: atributos, acción de generación y la salida que produce en el lenguaje destino.

## 1.1 Por qué la traducción necesita las otras dos capas

En Gleam `Int` y `Float` son tipos **disjuntos**, con operadores distintos: la suma de enteros es `+` y la de reales es `+.`. El operador de concatenación `<>` exige `String` en los dos lados. Eso significa que la fase SDT **no puede** decidir qué emitir mirando solo la forma de la expresión: necesita saber el tipo de cada operando.

Ese tipo lo resolvió el análisis semántico y llega a la fase SDT por dos caminos: la **tabla de símbolos**, que el traductor recibe ya construida, y el atributo sintetizado `type`, que el analizador anotó sobre cada nodo de expresión del árbol. Las tres capas del documento no son tres documentos pegados: la azul se apoya en la ámbar, que se apoya en la roja.

## 1.2 Resumen de las reglas semánticas (fondo ámbar)

| Regla | Nodos de la gramática | Qué valida |
|---|---|---|
| SEM-1 | <sent_declaracion>, <tipo_opcional>, <expresion> | Declaración de variable y compatibilidad de tipos |
| SEM-2 | <expr_primaria> (IDENTIFICADOR), <sent_lectura> | Uso de identificadores: declaración previa y visibilidad |
| SEM-3 | <expr_aditiva>, <expr_multiplicativa>, <expr_potencia>, <expr_unaria>, <expr_relacional>, <expr_igualdad>, <expr_y>, <expr_o>, <expr_concat> | Compatibilidad de operandos en operadores |
| SEM-4 | <sent_si>, <sent_mientras>, <sent_repita> | Condición de control de tipo booleano |
| SEM-5 | <sent_llamada>, <sufijo_id> dentro de <expr_primaria>, <argumentos> | Llamada a función: existencia, aridad y tipos |
| SEM-6 | <sent_mientras>, <sent_repita> | Terminación del bucle: la condición tiene que poder cambiar |
| SEM-7 | <sent_para>, y dentro de su <bloque>: <sent_declaracion>, <sent_lectura>, <sent_asignacion> | La variable de control del pa_cada es de solo lectura |
| SEM-8 | <sent_para>, <paso_opcional> | Paso del pa_cada distinto de cero y recorrido alcanzable |

## 1.3 Resumen de las reglas de traducción (azul)

| Regla | Producciones que cubre | Qué genera |
|---|---|---|
| SDT-1 | <programa>, <lista_declaraciones>, <declaracion> | Estructura del módulo de salida |
| SDT-2 | <def_funcion>, <parametros>, <param_lista>, <param_resto>, <tipo_retorno> | Definición de función |

| | | |
|---|---|---|
| SDT-3 | <sent_declaracion>, <tipo_opcional> | Declaración de variable |
| SDT-4 | <sent_lectura> | Lectura de consola, con conversión de tipo |
| SDT-5 | <sent_impresion> | Impresión por consola |
| SDT-6 | <sent_si>, <rama_sino>, <sent_retornar> | Condicional y reestructuración del retorno anticipado |
| SDT-7 | <sent_mientras> | Bucle de condición al principio → recursión de cola |
| SDT-8 | <sent_para>, <paso_opcional> | Bucle con contador → list.range o recursión |
| SDT-9 | <sent_repita> | Bucle de condición al final → cuerpo antes del case |
| SDT-10 | <sent_pillemos>, <lista_casos>, <lista_casos_resto>, <caso>, <patron>, <cuerpo_caso> | Calce de patrones |
| SDT-11 | <expr_aditiva>, <expr_multiplicativa>, <expr_potencia>, <expr_relacional>, <expr_igualdad>, <expr_concat>, <expr_y>, <expr_o>, <expr_unaria> | Operadores: la elección depende del tipo |
| SDT-12 | <lista_declaraciones>, <lista_sentencias>, <param_resto>, <arg_resto>, <lista_casos_resto>, <rama_sino>, <paso_opcional>, <tipo_opcional>, <expr_o_p> … <expr_potencia_p> | Listas vacías (ε) y producciones recursivas |
| SDT-13 | <sent_llamada>, <argumentos>, <arg_resto>, <sufijo_id>, <bloque> | Llamada a función y cierre de bloque |

Las trece reglas SDT cubren todas las producciones que contribuyen a la salida. **SDT-12** atiende de forma explícita lo que el enunciado exige sobre las listas vacías (producciones ε) y las producciones recursivas, que en esta gramática aparecen en todas las listas y en las colas de expresión.

# 2. Producciones BNF con las reglas intercaladas

Se reproduce la sección de producciones de la gramática. Cada regla aparece inmediatamente después del nodo que restringe o traduce.

```
  Ordenadas de <programa> (arriba) a las expresiones atomicas (abajo).
  Sin recursividad por la izquierda y factorizadas por la izquierda.
  Los terminales literales van entre comillas o en minuscula sin <>.
  e denota la cadena vacia (epsilon).
================================================================================
/* ------------------------------------------------------------------ */
/* 3.1  Programa y declaraciones                                      */
/* ------------------------------------------------------------------ */

<programa>              ::= <lista_declaraciones> $
```

### REGLA SDT-1 — Estructura del módulo de salida

Nodos de la gramática: <programa>, <lista_declaraciones>, <declaracion>

| | |
|---|---|
| **Atributos** | **modulos**: *sintetizado* acumulado en todo el recorrido. Conjunto de módulos de la biblioteca estándar de Gleam que las demás acciones SDT pidieron (`gleam/io`, `gleam/int`, …).<br>**code(decl)**: *sintetizado* en cada <declaracion>. |
| **Acción de traducción** | `programa.code ← imports(modulos) ‖ funciones ‖ main`<br>1. Separar las <declaracion> en definiciones de función y sentencias de nivel superior.<br>2. Gleam no admite sentencias fuera de una función: las de nivel superior se envuelven en `pub fn main() { … }`.<br>3. Emitir los `import` al principio, ordenados, solo los que alguna acción haya pedido.<br>4. Añadir al final las funciones auxiliares generadas por los bucles. |
| **Salida en Gleam** | `import gleam/io`<br>`import gleam/int`<br><br>`pub fn main() { … }`<br><br>`pub fn f(…) { … }`<br><br>`fn bucle_1(…) { … }` |

```
<lista_declaraciones>   ::= <declaracion> <lista_declaraciones>
                          | e

<declaracion>           ::= <def_funcion>
                          | <sentencia>

/* ------------------------------------------------------------------ */
/* 3.2  Definicion de funciones                                       */
/* ------------------------------------------------------------------ */

<def_funcion>           ::= hagale_pues IDENTIFICADOR
                            "(" <parametros> ")"
                            <tipo_retorno>
                            dele_pues
                                <bloque>
                            ya_quedo
```

## REGLA SDT-2 — Definición de función

Nodos de la gramática: <def_funcion>, <parametros>, <param_lista>, <param_resto>, <tipo_retorno>

| | |
|---|---|
| **Atributos** | **nombre**: *heredado* del token IDENTIFICADOR.<br>**params**: *sintetizado* recorriendo <param_lista> y su cola recursiva <param_resto>. Lista de pares (nombre, tipo).<br>**tipo_retorno**: *sintetizado* en <tipo_retorno>; vale ⊥ cuando la producción deriva en ε.<br>**code(bloque)**: *sintetizado* en el cuerpo. |
| **Acción de traducción** | `code ← "pub fn " nombre "(" params ") -> " T " {" code(bloque) "}"`<br>1. Traducir cada tipo de Paisascript al de Gleam: numerito→Int, quebradito→Float, cuento→String, siono→Bool.<br>2. Si **tipo_retorno** = ⊥ se omite la flecha y Gleam infiere el tipo.<br>3. Registrar los parámetros como variables vivas antes de traducir el cuerpo, y restaurar el entorno al salir.<br>4. Cerrar el cuerpo con una expresión (ver SDT-11). |
| **Salida en Gleam** | `hagale_pues f(numerito n) pa_que_lleve numerito dele_pues … ya_quedo`<br>se convierte en<br>`pub fn f(n: Int) -> Int { … }` |

```
<tipo_retorno>          ::= pa_que_lleve <tipo>
                          | e

<parametros>            ::= <param_lista>
                          | e

<param_lista>           ::= <tipo> IDENTIFICADOR <param_resto>

<param_resto>           ::= "," <tipo> IDENTIFICADOR <param_resto>
                          | e

<tipo>                  ::= numerito
                          | quebradito
                          | cuento
                          | siono

/* ------------------------------------------------------------- */
/* 3.3  Bloques y sentencias                                     */
/* ------------------------------------------------------------- */

<bloque>                ::= <sentencia> <lista_sentencias>

<lista_sentencias>      ::= <sentencia> <lista_sentencias>
                          | e
```

## REGLA SDT-12 — Listas vacías (ε) y producciones recursivas

Nodos de la gramática: <lista_declaraciones>, <lista_sentencias>, <param_resto>, <arg_resto>, <lista_casos_resto>, <rama_sino>, <paso_opcional>, <tipo_opcional>, <expr_o_p> ... <expr_potencia_p>

| | |
|---|---|
| **Atributos** | **code**: *sintetizado* en cada nodo de la lista.<br>**heredado_izq**: en las colas de expresión (<expr_X_p>), el código y el tipo del operando izquierdo bajan como atributo *heredado*, porque la gramática está factorizada a la derecha para ser LL(1). |
| **Acción de traducción** | La gramática es recursiva por la derecha, así que cada lista tiene una producción vacía. Las reglas son uniformes:<br>**Producción ε** → `code ← ""`. No emite nada. En <rama_sino> vacía la acción emite `False -> Nil`, porque un `case` de Gleam debe cubrir todos los casos. En <paso_opcional> vacío, `paso ← 1`. En <tipo_opcional> vacío el tipo se infiere de la expresión.<br>**Producción recursiva** → `code ← code(elemento) ‖ code(resto)`: se concatena el elemento con el resultado de la cola, separando con salto de línea las sentencias y con coma los parámetros y argumentos.<br>**Colas de expresión** → `A' ::= op B A'` se resuelve pasando el resultado parcial hacia la derecha como atributo heredado, de modo que `a - b - c` se asocie a la izquierda, como corresponde. |
| **Salida en Gleam** | `<lista_sentencias> ::= ε` → no emite nada.<br>`<rama_sino> ::= ε` → `False -> Nil`<br>`<paso_opcional> ::= ε` → paso 1 → `list.range(a, b)` |

```
<sentencia>             ::= <sent_declaracion>
                          | <sent_lectura>
                          | <sent_impresion>
                          | <sent_si>
                          | <sent_mientras>
                          | <sent_para>
                          | <sent_repita>
                          | <sent_pillemos>
                          | <sent_retornar>
                          | <sent_llamada>

/* Declaracion / asignacion. El tipo es opcional (se infiere).       */
<sent_declaracion>      ::= pille_pues <tipo_opcional> IDENTIFICADOR
                            "=" <expresion>
```

## REGLA SEM-1 — Declaración de variable y compatibilidad de tipos

Nodos de la gramática: <sent_declaracion>, <tipo_opcional>, <expresion>

| | |
|---|---|
| **Atributos involucrados** | **tipo_declarado**: *heredado* desde <tipo_opcional>. Dominio: {numerito, quebradito, cuento, siono} ∪ {⊥}, donde ⊥ = sin anotación.<br>**type(expr)**: *sintetizado* en <expresion>. Dominio: {numerito, quebradito, cuento, siono} ∪ {error}.<br>**tipo_simbolo**: *sintetizado* en <sent_declaracion>. Es el tipo con el que el identificador entra a la tabla de símbolos. |
| **Restricción** | Si **tipo_declarado** ≠ ⊥, entonces **type(expr) = tipo_declarado**.<br>Si **tipo_declarado** = ⊥, se infiere **tipo_simbolo = type(expr)** (como el *let* de Gleam). |
| **Acciones del compilador** | 1. Calcular type(expr) recorriendo la expresión (atributo sintetizado).<br>2. Comparar con tipo_declarado. Si difieren, reportar error con **fila, columna, lexema** y mensaje descriptivo.<br>3. Insertar el símbolo en el **ámbito actual** con su tipo, estado de inicialización = verdadero y posición de declaración.<br>4. El shadowing (redeclaración en el mismo ámbito) es válido: crea un enlace nuevo, no modifica el anterior.<br>5. Continuar el análisis (recuperación). |

## REGLA SDT-3 — Declaración de variable

Nodos de la gramática: <sent_declaracion>, <tipo_opcional>

| | |
|---|---|
| **Atributos** | **id.lexema**: *heredado* del token.<br>**code(expr)** y **type(expr)**: *sintetizados* en la expresión; el tipo lo resolvió el análisis semántico y queda anotado en el nodo.<br>**entorno**: *heredado*; la tabla de tipos vivos que la fase SDT va alimentando. |
| **Acción de traducción** | `code ← "let " id.lexema " = " code(expr)`<br>1. Emitir el enlace con `let`.<br>2. Registrar `entorno[id] ← type(expr)`, porque las acciones posteriores necesitan ese tipo para elegir operadores.<br>3. La redeclaración del mismo nombre es válida: en Gleam `let` crea un enlace nuevo (shadowing), que es justo la semántica de Paisascript. |
| **Salida en Gleam** | `pille_pues numerito x = 5 → let x = 5` |

```
<tipo_opcional>          ::= <tipo>
                           | e
<sent_lectura>           ::= escuche_pues "(" IDENTIFICADOR ")"
```

## REGLA SEM-2 — Uso de identificadores: declaración previa y visibilidad

Nodos de la gramática: <expr_primaria> (IDENTIFICADOR), <sent_lectura>

| | |
|---|---|
| **Atributos involucrados** | **id.lexema**: *heredado* del token IDENTIFICADOR.<br>**id.type**: *sintetizado* desde la tabla de símbolos. Dominio: {numerito, quebradito, cuento, siono, desconocido} ∪ {error}.<br>**id.declarado**: *sintetizado*. Dominio: {verdadero, falso}. |
| **Restricción** | **id.declarado = verdadero**: todo identificador usado en una expresión debe existir en el ámbito actual o en alguno de sus ancestros (alcance léxico anidado).<br>**escuche_pues(id)** declara el identificador si no existe y lo marca como inicializado. |
| **Acciones del compilador** | 1. Recorrer la pila de ámbitos de dentro hacia afuera buscando el lexema (resolución de nombres).<br>2. Si no aparece, reportar «identificador no declarado» con fila, columna y lexema.<br>3. Si aparece, sintetizar id.type con el tipo del símbolo; si es una función usada sin paréntesis, reportarlo.<br>4. En <sent_lectura>, declarar/inicializar el identificador y continuar. |

## REGLA SDT-4 — Lectura de consola, con conversión de tipo

Nodos de la gramática: <sent_lectura>

| | |
|---|---|
| **Atributos** | **id.lexema**: *heredado*.<br>**id.type**: consultado en la **tabla de símbolos** que construyó el análisis semántico. |
| **Acción de traducción** | `code ← "let assert Ok(" id ") = erlang.get_line(\"\")"`<br>1. Pedir el módulo `gleam/erlang`.<br>2. La consola entrega **String**. Si la tabla de símbolos dice que la variable es numerito o quebradito, añadir la conversión, porque Gleam no convierte de forma implícita:<br>`let assert Ok(x) = int.parse(string.trim(x))`<br>3. Si el tipo es desconocido, la variable queda como cuento. |
| **Salida en Gleam** | `escuche_pues(edad)` con `edad: numerito` →<br>`let assert Ok(edad) = erlang.get_line("")`<br>`let assert Ok(edad) = int.parse(string.trim(edad))` |

```
<sent_impresion>         ::= hable_pues "(" <expresion> ")"
```

## REGLA SDT-5 — Impresión por consola

Nodos de la gramática: <sent_impresion>

| | |
|---|---|
| **Atributos** | **code(expr)** y **type(expr)**: *sintetizados.* |
| **Acción de traducción** | `code ← "io.println(" aCadena(code(expr), type(expr)) ")"`<br>La función auxiliar `aCadena` es la misma de SDT-9: `io.println` solo acepta String, así que un numerito pasa por `int.to_string`, un quebradito por `float.to_string` y un siono por `bool.to_string`. |
| **Salida en Gleam** | `hable_pues(total)` con total numerito →<br>`io.println(int.to_string(total))` |

```
<sent_retornar>         ::= entregue_pues <expresion>

/* Llamada a funcion usada como sentencia (se descarta su valor).    */
<sent_llamada>          ::= IDENTIFICADOR "(" <argumentos> ")"
```

## REGLA SDT-13 — Llamada a función y cierre de bloque

Nodos de la gramática: <sent_llamada>, <argumentos>, <arg_resto>, <sufijo_id>, <bloque>

| | |
|---|---|
| **Atributos** | **nombre**: *heredado.*<br>**args**: *sintetizado* recorriendo la cola <arg_resto>.<br>**ultima**: *sintetizado* en <bloque>; indica si la última línea del bloque es una expresión o un enlace. |
| **Acción de traducción** | `code ← nombre "(" args ")"`, con los argumentos separados por coma. Una llamada vale igual como sentencia que como expresión, porque en Gleam todo es expresión.<br>**Cierre de bloque**: en Gleam todo bloque DEVUELVE un valor, así que no puede terminar en `let`. Si la última línea traducida es un enlace, la acción añade `Nil`, que es el valor unitario. Sin esto el código generado no compilaría. |
| **Salida en Gleam** | `sumar(1, 2)` → `sumar(1, 2)`<br>un bloque que termina en `let x = 5` recibe una línea `Nil` al final. |

```
/* ---------------------------------------------------------------- */
/* 3.4  Condicional  (sin recursividad izquierda)                   */
/* ---------------------------------------------------------------- */

<sent_si>               ::= si_acaso <expresion> entonces_pues
                                <bloque>
                            <rama_sino>
                            asi_quedo
```

## REGLA SDT-6 — Condicional y reestructuración del retorno anticipado

Nodos de la gramática: <sent_si>, <rama_sino>, <sent_retornar>

| | |
|---|---|
| **Atributos** | **code(cond)**, **code(entonces)**, **code(sino)**: *sintetizados*.<br>**continuacion**: *heredado*. Es la lista de sentencias que vienen DESPUÉS del condicional. Es el atributo que hace posible eliminar el retorno anticipado. |
| **Acción de traducción** | Caso general:<br>`code ← "case " code(cond) " { True -> {" entonces "} False -> {" sino "} }"`<br>Caso del **retorno anticipado** (un `si_acaso` sin `sino_pues` cuyo cuerpo termina en `entregue_pues`): Gleam no tiene `return`, así que la rama False recibe la **continuación** completa.<br>Sin esta transformación las dos ramas del `case` tendrían tipos distintos y el código no compilaría. |
| **Salida en Gleam** | `si_acaso n < 2 entonces_pues entregue_pues naranjas asi_quedo`<br>`… resto …`<br>se convierte en<br>`case n < 2 {`<br>`  True -> { False }`<br>`  False -> { … resto … }`<br>`}` |

```
<rama_sino>             ::= sino_pues <bloque>
                          | e

/* ------------------------------------------------------------- */
/* 3.5  Repeticion por condicion  (while)                        */
/* ------------------------------------------------------------- */

<sent_mientras>         ::= mientras_que <expresion> dele_pues
                                <bloque>
                            hasta_ahi
```

## REGLA SEM-4 — Condición de control de tipo booleano

Nodos de la gramática: <sent_si>, <sent_mientras>, <sent_repita>

| | |
|---|---|
| **Atributos involucrados** | **type(cond)**: *sintetizado* en la expresión de control. Dominio: tipos del lenguaje ∪ {desconocido, error}.<br>**cond_esperado**: *heredado* desde la construcción. Dominio: {siono}. |
| **Restricción** | **type(cond) = siono** en **si_acaso**, en **mientras_que** y en el **hasta_que** de **repita_pues**. Una condición numerito, cuento o quebradito no es válida.<br>En **repita_pues** la condición se evalúa en el ámbito del cuerpo, así que puede leer las variables que el cuerpo creó. |
| **Acciones del compilador** | 1. Evaluar el tipo sintetizado de la condición.<br>2. Compararlo con siono.<br>3. Si no coincide, reportar «la condición debe ser siono» con fila, columna y lexema.<br>4. Analizar igualmente ambos bloques (recuperación). |

## REGLA SDT-7 — Bucle de condición al principio → recursión de cola

Nodos de la gramática: <sent_mientras>

| | |
|---|---|
| **Atributos** | **estado**: *sintetizado* del cuerpo. Variables que el cuerpo vuelve a enlazar Y que ya existían fuera: son las únicas que cambian de una vuelta a otra.<br>**libres**: *sintetizado*. Variables que el bucle solo LEE y vienen de fuera.<br>**tipos**: de la **tabla de símbolos**, para anotar la firma.<br>**continuacion**: *heredado* (ver SDT-6). |
| **Acción de traducción** | Gleam no tiene `while`: se genera una función auxiliar recursiva de cola, que el BEAM ejecuta en espacio constante.<br>1. Calcular **estado** y **libres**. Los dos entran como parámetros; solo el estado se devuelve.<br>2. Emitir `fn bucle_k(libres, estado) -> T { case cond { True -> { cuerpo; bucle_k(…) } False -> estado } }`.<br>3. En el punto de la llamada: `let estado = bucle_k(libres, estado)`.<br>4. Si el cuerpo contiene un `entregue_pues`, la rama False recibe la continuación y la auxiliar devuelve el valor de la función que la contiene. |
| **Salida en Gleam** | `mientras_que i <= n dele_pues … hasta_ahi →`<br>`fn bucle_1(n: Int, acumulado: Int, i: Int) -> #(Int, Int) {`<br>`  case i <= n {`<br>`  True -> { … bucle_1(n, acumulado, i) }`<br>`  False -> #(acumulado, i)`<br>`  }`<br>`}` |

```
/* ------------------------------------------------------------- */
/* 3.6  Repeticion con variable de control  (for)                */
/* ------------------------------------------------------------- */

<sent_para>             ::= pa_cada IDENTIFICADOR
                            desde <expresion>
                            hasta <expresion>
                            <paso_opcional>
                            dele_pues
                                <bloque>
                            listo_pues
```

## REGLA SEM-7 — La variable de control del pa_cada es de solo lectura

Nodos de la gramática: <sent_para>, y dentro de su <bloque>: <sent_declaracion>, <sent_lectura>, <sent_asignacion>

| | |
|---|---|
| **Atributos involucrados** | **var_control.lexema**: *heredado* del IDENTIFICADOR que sigue a `pa_cada`.<br>**var_control.escribible**: *heredado* hacia el ámbito del bucle. Dominio: {verdadero, falso}; vale **falso** para la variable de control y verdadero para cualquier otra.<br>**var_control.categoria**: *sintetizado* al declararla. Dominio: {bucle}.<br>**var_control.type**: *sintetizado* = numerito. |
| **Restricción** | Ninguna sentencia del cuerpo puede volver a enlazar la variable de control: ni `pille_pues i = …`, ni `escuche_pues(i)`, ni `i = …`.<br>El atributo se consulta antes de cada redefinición: solo se admite si **escribible = verdadero**.<br>Fuera del bucle el nombre vuelve a estar libre, porque el ámbito del `pa_cada` se cierra al terminar. |
| **Acciones del compilador** | 1. Al entrar al `pa_cada`, abrir un ámbito nuevo e insertar la variable de control con categoría = bucle, tipo = numerito, inicializada = verdadero y **escribible = falso**.<br>2. En cada `pille_pues` y cada `escuche_pues` del cuerpo, buscar el nombre en la pila de ámbitos y leer su atributo escribible.<br>3. Si es falso, reportar error con fila, columna y lexema, y sugerir otro nombre para la variable nueva.<br>4. Cerrar el ámbito al salir del bucle y continuar.<br>*Por qué importa para Gleam:* la variable de control se vuelve el parámetro de `fn(i) { … }` dentro de `list.each`; reenlazarla rompería el recorrido. |

## REGLA SDT-8 — Bucle con contador → list.range o recursión

Nodos de la gramática: <sent_para>, <paso_opcional>

| | |
|---|---|
| **Atributos** | **var**: *heredado*, la variable de control.<br>**code(desde)**, **code(hasta)**: *sintetizados*.<br>**paso**: *sintetizado* de <paso_opcional>; vale 1 cuando la producción deriva en ε (ver SDT-12). |
| **Acción de traducción** | Hay dos caminos, y la elección depende del paso:<br>**paso = 1** → la forma idiomática de Gleam:<br>`list.range(desde, hasta) \|> list.each(fn(var) { cuerpo })`<br>Pide el módulo `gleam/list`.<br>**paso ≠ 1** → `list.range` de Gleam NO admite incremento, así que se genera recursión de cola con la comparación orientada según el signo del paso (`<=` si avanza, `>=` si retrocede). |
| **Salida en Gleam** | `pa_cada k desde 0 hasta 10 de_a 2 dele_pues … listo_pues` →<br>`fn bucle_2(k: Int) -> Nil {`<br>`  case k <= 10 {`<br>`  True -> { … bucle_2(k + 2) }`<br>`  False -> Nil`<br>`  }`<br>`}` |

```
<paso_opcional>          ::= de_a <expresion>
                           | e
```

## REGLA SEM-8 — Paso del pa_cada distinto de cero y recorrido alcanzable

Nodos de la gramática: <sent_para>, <paso_opcional>

| | |
|---|---|
| **Atributos involucrados** | **valor(desde)**, **valor(hasta)**, **valor(paso)**: *sintetizados* por plegado de constantes sobre cada <expresion>. Dominio: $\mathbb{Z} \cup \{\bot\}$, donde ⊥ significa «no es un literal» (admite el menos unario y los paréntesis).<br>**paso_implicito**: *heredado*; vale 1 cuando <paso_opcional> deriva en ε.<br>**direccion**: *sintetizado* en <sent_para> = signo(valor(paso)). Dominio: {+1, −1}. |
| **Restricción** | **valor(paso) ≠ 0**: con paso 0 la variable de control nunca avanza.<br>Si **valor(paso) > 0** entonces **valor(desde) ≤ valor(hasta)**.<br>Si **valor(paso) < 0** entonces **valor(desde) ≥ valor(hasta)**.<br>Las dos últimas solo se exigen cuando los tres valores son literales: con límites variables el recorrido no es decidible en tiempo de compilación y la regla calla en lugar de dar un falso positivo. |
| **Acciones del compilador** | 1. Plegar las tres expresiones a su valor constante; si <paso_opcional> es ε, usar paso_implicito = 1.<br>2. Si valor(paso) = 0, reportar error sobre el lexema del paso con fila y columna.<br>3. Si los tres son literales, comparar el signo del paso con el orden de los límites y reportar «el cuerpo nunca se ejecuta» indicando los tres valores concretos.<br>4. Si alguno no es literal, no reportar nada y seguir.<br>5. Continuar con el cuerpo del bucle.<br>*Por qué importa para Gleam:* `list.range` no admite paso, así que con paso ≠ 1 el generador emite recursión de cola; un paso 0 produciría una recursión infinita. |

```
/* ------------------------------------------------------------- */
/* 3.7  Repeticion con condicion al final  (do-while / repeat-until)*/
/* ------------------------------------------------------------- */

<sent_repita>            ::= repita_pues
                                 <bloque>
                             hasta_que <expresion>
```

## REGLA SEM-6 — Terminación del bucle: la condición tiene que poder cambiar

Nodos de la gramática: <sent_mientras>, <sent_repita>

| | |
|---|---|
| **Atributos involucrados** | **leidas(cond)**: *sintetizado* en la expresión de control. Dominio: conjunto de identificadores que la condición consulta (sin los nombres de función).<br>**redefinidas(cuerpo)**: *sintetizado* en <bloque>. Dominio: conjunto de identificadores que el cuerpo vuelve a enlazar con `pille_pues` o `escuche_pues`.<br>**valor(cond)**: *sintetizado* por plegado de constantes. Dominio: {sizas, naranjas} ∪ {⊥} (⊥ = no constante).<br>**continua_si**: *heredado* de la construcción. Dominio: {verdadero} para `mientras_que`, {falso} para `repita_pues`, porque `hasta_que` es la condición de SALIDA, no de permanencia. |
| **Restricción** | **valor(cond) ≠ continua_si**: la condición no puede ser el literal que mantiene el bucle vivo (`mientras_que sizas` o `hasta_que naranjas`).<br>**leidas(cond) ≠ ∅**: una condición sin variables tiene siempre el mismo valor.<br>**leidas(cond) ∩ redefinidas(cuerpo) ≠ ∅**: el cuerpo debe redefinir al menos una de las variables que la condición lee.<br>La regla es *conservadora y sólida en este lenguaje*: Paisascript es inmutable y sin efectos laterales, así que si esa intersección es vacía ninguna ejecución puede cambiar el valor de la condición. |
| **Acciones del compilador** | 1. Plegar la condición: si es un literal booleano, compararlo con continua_si y reportar «el bucle nunca termina».<br>2. Recorrer la condición y recolectar leidas(cond), descartando los identificadores que la tabla de símbolos marca como función.<br>3. Recorrer el cuerpo (incluidos los bucles anidados) y recolectar redefinidas(cuerpo) a partir de los nodos <sent_declaracion>, <sent_lectura> y <sent_asignacion>.<br>4. Intersecar ambos conjuntos. Si el resultado es vacío, reportar error con fila, columna y lexema del bucle, nombrando las variables de la condición que habría que hacer avanzar.<br>5. Continuar el análisis del resto del programa.<br>*Por qué importa para Gleam:* ese bucle se traduce a una recursión de cola; sin progreso, la función auxiliar nunca alcanza su caso base. |

## REGLA SDT-9 — Bucle de condición al final → cuerpo antes del case

Nodos de la gramática: <sent_repita>

| | |
|---|---|
| **Atributos** | Los mismos de SDT-7: **estado**, **libres**, **continuacion**.<br>**sentido**: *heredado* de la construcción. En `repita_pues` la condición es de SALIDA, no de permanencia. |
| **Acción de traducción** | Es la misma recursión de cola de SDT-7 con dos diferencias, y las dos vienen de la semántica post-test:<br>1. El cuerpo se emite **ANTES** del `case`, no dentro de una de sus ramas. Eso es lo que hace que corra al menos una vez.<br>2. Las ramas se invierten: `True` (se cumplió `hasta_que`) devuelve el estado y es el caso base; `False` vuelve a llamar. |
| **Salida en Gleam** | `repita_pues … hasta_que v >= 3 →`<br>`fn bucle_3(v: Int) -> Int {`<br>`  … cuerpo …`<br>`  case v >= 3 {`<br>`  True -> v`<br>`  False -> bucle_3(v)`<br>`  }`<br>`}` |

```
/* El cuerpo se ejecuta AL MENOS UNA VEZ y hasta_que da la condicion de  */
/* SALIDA (no de permanencia): el bucle repite mientras la expresion sea */
/* falsa. hasta_que cierra el bloque, de modo que no hace falta una      */
/* palabra de cierre adicional. La condicion se evalua DENTRO del ambito */
/* del cuerpo, asi que puede leer las variables que el cuerpo acaba de   */
```

```
/* crear: es el mismo estado que el generador pasa a la recursion de     */
/* cola de Gleam.                                                        */
/* --------------------------------------------------------------------- */
/* 3.8  Calce de patrones  (equivale al `case` de Gleam)                 */
/* --------------------------------------------------------------------- */
<sent_pillemos>         ::= pillemos <expresion> "{" <lista_casos> "}"
```

### REGLA SDT-10 — Calce de patrones

Nodos de la gramática: <sent_pillemos>, <lista_casos>, <lista_casos_resto>, <caso>, <patron>, <cuerpo_caso>

| | |
|---|---|
| **Atributos** | **code(sujeto)**: *sintetizado*.<br>**code(patron)**: *sintetizado* en cada <patron>.<br>**casos**: *sintetizado* recorriendo la cola recursiva <lista_casos_resto> (ver SDT-12). |
| **Acción de traducción** | `code ← "case " code(sujeto) " { " casos " }"`<br>Es la traducción más directa del lenguaje, porque `pillemos` se diseñó contra el `case` de Gleam.<br>Conversión de patrones: `sizas`→`True`, `naranjas`→`False`, `_`→`_`, y los literales reales reciben punto decimal explícito.<br>Un cuerpo de una sola línea va en la misma línea del patrón; uno de varias va entre llaves. |
| **Salida en Gleam** | `pillemos x { 10 pa_que_lleve hable_pues("diez") }` →<br>`case x { 10 -> io.println("diez") }` |

```
<lista_casos>           ::= <caso> <lista_casos_resto>
<lista_casos_resto>     ::= <caso> <lista_casos_resto>
                          | e
<caso>                  ::= <patron> pa_que_lleve <cuerpo_caso>
<cuerpo_caso>           ::= "{" <bloque> "}"
                          | <sentencia>
<patron>                ::= NUM_ENTERO
                          | NUM_REAL
                          | CADENA_LITERAL
                          | sizas
                          | naranjas
                          | IDENTIFICADOR      /* enlaza el valor (catch-all) */
                          | "_"                /* comodin, descarta el valor  */
/* --------------------------------------------------------------------- */
/* 3.9  Expresiones                                                      */
/*      Jerarquia de precedencia, de menor a mayor:                      */
/*        o_que < y_tambien < igualdad < relacional < <>                 */
/*              < +,- < *,/,% < ** < unarios < primaria                  */
/*      Toda recursividad izquierda fue eliminada con la tecnica         */
/*      A ::= B A'   /   A' ::= op B A' | e                              */
/* --------------------------------------------------------------------- */
<expresion>             ::= <expr_o>
<expr_o>                ::= <expr_y> <expr_o_p>
<expr_o_p>              ::= o_que <expr_y> <expr_o_p>
                          | e
<expr_y>                ::= <expr_igualdad> <expr_y_p>
<expr_y_p>              ::= y_tambien <expr_igualdad> <expr_y_p>
                          | e
<expr_igualdad>         ::= <expr_relacional> <expr_igualdad_p>
<expr_igualdad_p>       ::= <op_igualdad> <expr_relacional> <expr_igualdad_p>
                          | e
<op_igualdad>           ::= igualito
                          | distinto
<expr_relacional>       ::= <expr_concat> <expr_relacional_p>
<expr_relacional_p>     ::= <op_relacional> <expr_concat> <expr_relacional_p>
                          | e
<op_relacional>         ::= ">"
                          | "<"
                          | ">="
                          | "<="
```

```
<expr_concat>           ::= <expr_aditiva> <expr_concat_p>
<expr_concat_p>         ::= "<>" <expr_aditiva> <expr_concat_p>
                          | e

<expr_aditiva>          ::= <expr_multiplicativa> <expr_aditiva_p>
<expr_aditiva_p>        ::= "+" <expr_multiplicativa> <expr_aditiva_p>
                          | "-" <expr_multiplicativa> <expr_aditiva_p>
                          | e

<expr_multiplicativa>   ::= <expr_potencia> <expr_multiplicativa_p>
<expr_multiplicativa_p> ::= "*" <expr_potencia> <expr_multiplicativa_p>
                          | "/" <expr_potencia> <expr_multiplicativa_p>
                          | "%" <expr_potencia> <expr_multiplicativa_p>
                          | e
```

## REGLA SEM-3 — Compatibilidad de operandos en operadores

Nodos de la gramática: <expr_aditiva>, <expr_multiplicativa>, <expr_potencia>, <expr_unaria>, <expr_relacional>, <expr_igualdad>, <expr_y>, <expr_o>, <expr_concat>

| | |
|---|---|
| **Atributos involucrados** | **type_izq** y **type_der**: *sintetizados* en los dos operandos. Dominio: tipos del lenguaje ∪ {desconocido, error}.<br>**type**: *sintetizado* en el nodo del operador (tipo del resultado). |
| **Restricción** | **+ − * / ****: type_izq = type_der y ambos ∈ {numerito, quebradito}.<br>**%**: type_izq = type_der = numerito.<br>**> < >= <=**: operandos numéricos del mismo tipo; el resultado es siono.<br>**y_tambien / o_que / nanai**: operandos siono; resultado siono.<br>**<>**: operandos cuento, numerito o quebradito; resultado cuento (Paisascript convierte a texto al generar Gleam). |
| **Acciones del compilador** | 1. Comparar type_izq con type_der según la tabla anterior.<br>2. Si no son compatibles, reportar el operador, los dos tipos y la posición (fila, columna, lexema).<br>3. Sintetizar el tipo del resultado o **error**; el tipo error es compatible con todo para no generar errores en cascada.<br>4. Continuar con el resto de la expresión. |

## REGLA SDT-11 — Operadores: la elección depende del tipo

Nodos de la gramática: <expr_aditiva>, <expr_multiplicativa>, <expr_potencia>, <expr_relacional>, <expr_igualdad>, <expr_concat>, <expr_y>, <expr_o>, <expr_unaria>

| | |
|---|---|
| **Atributos** | **type_izq** y **type_der**: *sintetizados* por el análisis semántico y anotados en el árbol. **Son el atributo crítico de toda la fase SDT.**<br>**code_izq**, **code_der**: *sintetizados*. |
| **Acción de traducción** | En Gleam `Int` y `Float` son tipos disjuntos con operadores **distintos**, así que la acción consulta el tipo antes de emitir:<br>`SI type ∈ {quebradito} → + - * / > < >= <= se emiten como +. -. *. /. >. <. >=. <=.`<br>`igualito → ==`, `distinto → !=`, `y_tambien → &&`, `o_que → ||`, `nanai → !`<br>** no es operador en Gleam: se emite `int.power` / `float.power`, que devuelven `Result`, envueltos en `result.unwrap`.<br><> exige String en ambos lados: la acción inserta `int.to_string` / `float.to_string` donde haga falta. |
| **Salida en Gleam** | `a + b` con quebradito → `a +. b`<br>`x + y` con numerito → `x + y`<br>`"n = " <> x` → `"n = " <> int.to_string(x)` |

```
/* La potencia asocia a la DERECHA: 2 ** 3 ** 2  =  2 ** (3 ** 2)   */
<expr_potencia>         ::= <expr_unaria> <expr_potencia_p>
<expr_potencia_p>       ::= "**" <expr_potencia>
                          | e

<expr_unaria>           ::= nanai <expr_unaria>
                          | "-" <expr_unaria>
                          | <expr_primaria>
```

```
<expr_primaria>          ::= NUM_ENTERO
                           | NUM_REAL
                           | CADENA_LITERAL
                           | sizas
                           | naranjas
                           | IDENTIFICADOR <sufijo_id>
                           | "(" <expresion> ")"

/* Factorizacion por la izquierda: variable y llamada comparten el   */
/* prefijo IDENTIFICADOR, por eso se separa el sufijo.               */
<sufijo_id>              ::= "(" <argumentos> ")"
                           | e

<argumentos>             ::= <arg_lista>
                           | e

<arg_lista>              ::= <expresion> <arg_resto>

<arg_resto>              ::= "," <expresion> <arg_resto>
                           | e
```

### REGLA SEM-5 — Llamada a función: existencia, aridad y tipos

Nodos de la gramática: <sent_llamada>, <sufijo_id> dentro de <expr_primaria>, <argumentos>

| | |
|---|---|
| **Atributos involucrados** | **arity**: *sintetizado* (número de argumentos).<br>**arg_type[1..n]**: *sintetizados* en cada <expresion>.<br>**param_type[1..n]** y **return_type**: consultados en la firma de la tabla de símbolos.<br>**type**: *sintetizado* en la llamada = return_type. |
| **Restricción** | La función debe estar declarada; **arity** = \|**params**\|; **arg_type[i]** = **param_type[i]** para todo i; el tipo del resultado es el tipo de retorno declarado o inferido. |
| **Acciones del compilador** | 1. Buscar la firma de la función en el ámbito global.<br>2. Si no existe, reportar «función no declarada» con su posición.<br>3. Comparar la aridad y reportar el número esperado y el recibido.<br>4. Comparar cada argumento con su parámetro formal y reportar los desajustes de tipo.<br>5. Sintetizar return_type y continuar. |

==========

# 3. El compilador: cuatro fases encadenadas

El compilador procesa la entrada en cuatro fases, cada una en su propio módulo, y cada una recibe exactamente lo que produjo la anterior.

| # | Fase | Módulo | Qué produce |
|---|---|---|---|
| 1 | Análisis léxico | `lexer.py` | Tokens con lexema, fila y columna. Reporta los no reconocidos. |
| 2 | Análisis sintáctico | `parser.py` | El CST según la gramática BNF. Recuperación de errores para seguir analizando. |
| 3 | Análisis semántico | `analizador_semantico.py` | Tabla de símbolos y validación de las ocho reglas. Anota el tipo sintetizado sobre cada nodo de expresión. |
| 4 | Traducción (SDT) | `traductor_sdt.py` | Recorre el CST aplicando las acciones SDT y emite el código Gleam. |

El encadenamiento lo gobierna `compilador.py`, que es el único punto donde se decide si hay salida:

```
from compilador import compilar

resultado = compilar(codigo_fuente)
if resultado.exitoso:
    print(resultado.codigo_gleam)
else:
    print(resultado.reporte())      # errores de las tres fases, unificados
```

## 3.1 Si hay errores, no hay salida

Es una regla dura del enunciado y está implementada como tal: mientras exista un error léxico, sintáctico o semántico, la fase SDT **no se ejecuta** y el compilador se limita a reportar. Un error sintáctico además detiene el análisis semántico, porque un árbol roto solo produciría errores falsos.

## 3.2 Reporte de errores unificado

Las tres fases de análisis entregan sus errores al mismo objeto `ErrorCompilacion`, con **fase, fila, columna, lexema y mensaje descriptivo**. Cada fase aplica recuperación, de modo que una sola ejecución reporte el mayor número posible de problemas.

```
--- Fase SEMÁNTICA (8) ---
  [SEMÁNTICA] Fila 13, Columna 21 sobre 'edad': [SEM-1] Se declara 'edad'
  como numerito, pero la expresión es de tipo cuento
  [SEMÁNTICA] Fila 33, Columna 1 sobre 'mientras_que': [SEM-6] El cuerpo de
  'mientras_que' no redefine ninguna variable de la condición (tope): el
  bucle nunca termina
```

## 3.3 La tabla de símbolos durante la fase SDT

El traductor recibe la tabla de símbolos ya construida y la consulta para los tipos de las variables y las firmas de las funciones. Es lo que le permite saber que `escuche_pues(edad)` necesita `int.parse` cuando `edad` es numerito, y que `a + b` debe emitirse como `a +. b` cuando los operandos son quebradito.

## 3.4 Un ejemplo completo

**Entrada en Paisascript:**

```
hagale_pues sumatoria(numerito n) pa_que_lleve numerito dele_pues
    pille_pues numerito acumulado = 0
```

```
    pille_pues numerito i = 1
    mientras_que i <= n dele_pues
        pille_pues acumulado = acumulado + i
        pille_pues i = i + 1
    hasta_ahi
    entregue_pues acumulado
ya_quedo

pille_pues numerito total = sumatoria(10)
hable_pues("La sumatoria de 1 a 10 es: " <> total)
```

**Salida que produce la fase SDT:**

```
import gleam/int
import gleam/io

pub fn main() {
  let total = sumatoria(10)
  io.println("La sumatoria de 1 a 10 es: " <> int.to_string(total))
}

pub fn sumatoria(n: Int) -> Int {
  let acumulado = 0
  let i = 1
  let #(acumulado, i) = bucle_1(n, acumulado, i)
  acumulado
}

fn bucle_1(n: Int, acumulado: Int, i: Int) -> #(Int, Int) {
  case i <= n {
    True -> {
      let acumulado = acumulado + i
      let i = i + 1
      bucle_1(n, acumulado, i)
    }
    False -> #(acumulado, i)
  }
}
```

Obsérvense tres decisiones de la fase SDT en este fragmento: el `int.to_string` que exige la concatenación (SDT-11), el parámetro `n` que viaja a la función auxiliar porque el bucle lo lee aunque no lo modifique (SDT-7), y la tupla `#(acumulado, i)` con el estado del bucle, que es lo que reemplaza a la mutación que Gleam no tiene.

## 3.5 Archivos de prueba

| Archivo | Qué contiene | Resultado esperado |
|---|---|---|
| `1_sin_errores.paisa` | Programa válido | Las cuatro fases corren y se produce la salida completa en Gleam. |
| `2_errores_semanticos.paisa` | 8 errores semánticos | Se reportan los ocho con fila, columna y lexema. **No se genera salida.** |
| `3_errores_sintacticos.paisa` | 5 errores sintácticos | Se reportan con recuperación. No se llega al análisis semántico ni se genera salida. |

# 4. Bonus — Validación del código generado con IA

**Modalidad C: combinación de validación y sugerencias.** Una vez que la fase SDT produce el código Gleam, ese texto se envía a un modelo de lenguaje que primero comprueba si es sintáctica y semánticamente correcto para Gleam y, si lo es, propone mejoras concretas.

Es una capa **adicional**: el compilador funciona completo sin ella, y si no hay clave de API o no hay red, la interfaz lo indica y el resto sigue funcionando igual.

## 4.1 El prompt

```
Eres un revisor experto del lenguaje Gleam.

Este codigo fue GENERADO automaticamente por un compilador que traduce
desde Paisascript. Revisa dos cosas y responde en espanol:

1. VALIDEZ: ¿es sintactica y semanticamente correcto en Gleam? Revisa en
   particular: que los tipos Int y Float no se mezclen (Gleam usa + y +.
   como operadores distintos), que todo bloque termine en una expresion y
   no en un 'let', que los modulos usados esten importados, y que las
   ramas de cada 'case' devuelvan el mismo tipo.
2. OPTIMIZACION: propon mejoras concretas e idiomaticas.

Codigo generado:
{codigo_gleam}
```

## 4.2 Flujo de integración

1. El compilador ejecuta las cuatro fases y produce el código Gleam.
2. La interfaz ofrece el botón «Validar y optimizar con IA».
3. `ai_assistant.validar_codigo_generado(codigo_gleam)` construye el prompt y llama a la API.
4. La respuesta se muestra junto al código, separada de él y claramente etiquetada como sugerencia.